RFC

[Pipeline de Análisis Inteligente]

Request for Comments · Documento de diseño técnico

Campo	Valor
Identificador	RFC-[MEDIA]-[001]
Estado	Aprobado
Autor(es)	Juan Pablo García Ruiz Cerda
Revisores	Instructor del Curso
Fecha de creación	22/10/2026
Última actualización	25/10/2026
Versión	0.1
Sistema / componente	pipeline-llm

Instrucciones: reemplaza los campos entre corchetes, elimina las secciones que no apliquen y registra las decisiones relevantes.

1. Resumen ejecutivo

MediaGroup requiere automatizar sus procesos para el análisis de artículos periodísticos extrayendo metadatos claves (categoría, sentimiento y resumen SEO) de forma rápida y eficaz. Este documento propone una arquitectura en el servicio Google Cloud Plataform (GCP) basada en un Agente Inteligente (Google ADK) que sirve fundamentalmente para el procesamiento de textos en forma asíncrona.

Como solución se propone el uso de FastAPI en Clod Run para la API, Vertex AI para la inferencia de un modelo LLM (Gemini) y una base de datos Firebase para documentar métricas por cada consulta realizada en el pipeline. Se espera como resultado que el sistema sea escalable, seguro y económico.

2. Contexto y problema

Actualmente en MediaGroup Latam el tiempo para el análisis de artículos es tardado especialmente para la categorización y la resumición SEO de estos, por consiguiente, esto causa un bloqueo en el flujo de trabajo además del tiempo de entrega en distintas áreas.

2.1 Estado actual

El flujo funciona como un pipeline de datos en un archivo de Python, se ingestan webhooks en el modelo LLM (Gemini) que genera diferentes timeouts pero con errores, esto debido a que el tiempo de espera y la latencia es mayor al esperado, lo que puede generar problemas en las solicitudes.

2.2 Motivación

Se requiere de una arquitectura que responda a una latencia menor en el webhook (procesamiento asíncrono), que utilice un framework de agentes que permita razonamiento (ciclo ReAct) para mejorar la precisión del análisis, administrar métricas importantes y acelerar procesos de manera rápida, económica y segura.

3. Objetivos y alcance

3.1 Objetivos

- Reducir el tiempo de respuesta del webhook a menos de 200 ms
- Implementar un agente ADK con capacidad de usar herramientas (Tools) para analizar texto.
- Garantizar la administración de procesos y costos mediante la observabilidad de logs estructurados y el caculo de costos por consulta de articulo.

3.2 Dentro del alcance

- API REST en FastAPI con autenticación JWT y Rate Limiting.
- Integración con Vertex AI (Modelo Gemini 3.5 Flash en endpoint global).
- Frontend en React con visualización del ciclo ReAct y Dashboard de métricas.
- Despliegue en Cloud Run y configuración de Secret Manager.
- En caso de fallo en el modelo, el backend o la base de datos asegurar un Circuit Breaker.
- Controlar el número máximo de solicitudes mediante un rate limit.

3.3 Fuera del alcance

- Entrenamiento de modelos personalizados (Fine-tuning)
- Autenticación Oauth2 con proveedores externos (solo JWT local)
- Carga de documentos (pdf, xslx, png, jpg) en el agente.

4. Requisitos

4.1 Requisitos funcionales

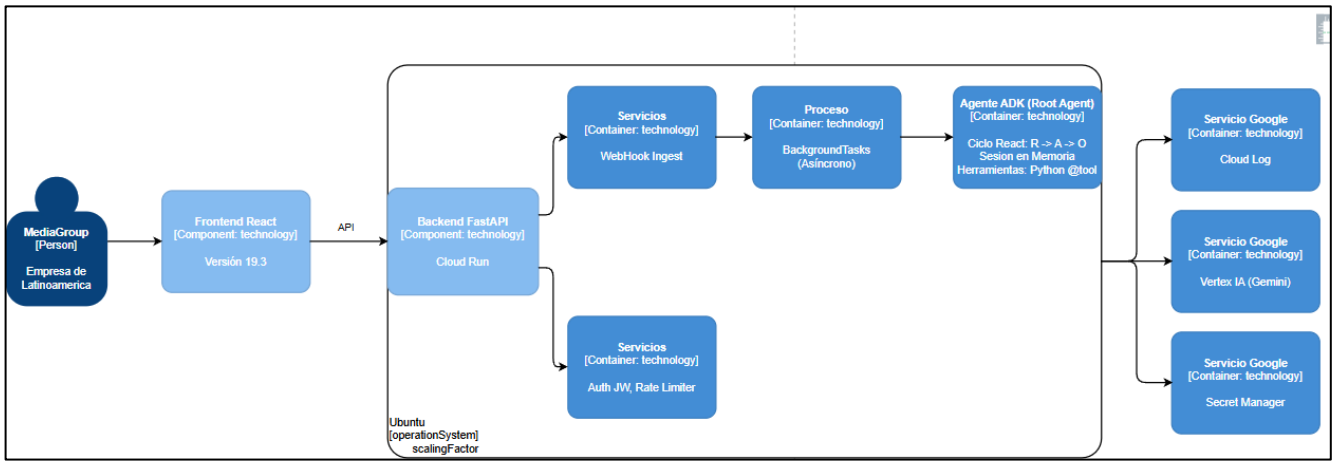
ID	Requisito	Prioridad	Criterio de aceptación
RF-01	El sistema debe recibir artículos vía webhook y encolarlos asíncronamente.	Alta	El endpoint responde HTTP 200 en < 200ms con un task_id.
RF-02	El Agente debe usar Vertex AI para extraer categoría, sentimiento y resumen SEO.	Alta	El log PIPELINE_COMPLETED contiene un JSON con las 3 claves.
RF-03	El Dashboard debe mostrar métricas reales de los artículos procesados.	Media	La interfaz consulta /dashboard/metrics y pinta los datos de Firestore.
R-04	El sistema debe autenticar correctamente las credenciales del usuario.	Alta	El Login valida el nombre de usuario y contraseña, en caso de ser errónea muestra un error de usuario o contraseña incorrectos.
R-05	El servicio de Google debe funcionar correctamente con el servidor local.	Media	Cuando el usuario interactúa con el agente este debe procesar exitosamente el pipeline en el log de Google. Así mismo al ejecutar el desplegué de cloud en el backend, este debe pasar por todos los procesos correctamente.

4.2 Requisitos no funcionales

Categoría	Requisito / objetivo	Método de validación
Rendimiento	Latencia del webhook < 200ms. Uso de BackgroundTasks	Prueba de carga con Postman/Swagger.
Seguridad	Autenticacion JWT. USO DE Secret Manager para claves	Revision de codigo y configuración de IAM.
Disponibilidad	El Sistema debe escalar a cero en inactividad (Cloud Run)	Monitoreo de instancias en GCP.
Mantenibilidad	Logging estructurado en JSON con trazabilidad de costos	Revision de Logs en Cloud Logging.

5. Propuesta de solución

5.1 Diseño de alto nivel



- **Frontend (React):** Interfaz de usuario para interactuar con el agente.
- **Backend (FastAPI en Cloud Run):** Recibe las peticiones, valida JWT y deriva el texto a un proceso en segundo plano.
- **Agente ADK (Root Agent):** Vive dentro del backend. Ejecuta el ciclo ReAct (Reason -> Act -> Observe) usando la herramienta analizar_con_llm.
- **Servicios GCP:** Vertex AI (inferencia), Firestore (base de datos), Secret Manager (credenciales), Cloud Log (monitoreo).

5.2 Componentes y responsabilidades

Componente	Responsabilidad	Tecnología / interfaz
API Backend	Recibir peticiones, autenticar y encolar.	FastAPI, Python 3.12
Agente ADK	Razonar y llamar a las herramientas de análisis.	Google ADK, LlmAgent
Base de Datos	Guardar historial y métricas de costos.	Google Cloud Firestore
Frontend	Interfaz gráfica de usuario.	React, Vite, Tailwind CSS

5.3 Flujo principal

1. El usuario se autentica en el Login.
2. Secret Manager almacena el JWT.
3. El usuario envía un texto desde el Chat (Frontend).
4. El Frontend hace un POST a /webhook/ingest con el token JWT.
5. El Backend valida el token, genera un task_id y lanza process_article_pipeline en BackgroundTasks. Responde HTTP 200 inmediatamente.
6. El Agente ADK invoca la Tool analizar_con_llm que llama a Vertex AI (endpoint global).
7. Vertex AI devuelve el JSON y el backend lo guarda en Firestore.
8. El Frontend (haciendo Polling a /webhook/result/{task_id}) obtiene el resultado y lo muestra en pantalla.
9. El Backend (llama a la API y la base de datos Firebase) para almacenar las métricas del resultado.
10. El usuario entra al DashBoard.
11. DashBoard muestra métricas de monitoreo de acuerdo al ultimo task_id.

6. Diseño de interfaces y datos

6.1 API / contratos

Elemento	Detalle
Método y ruta	POST /api/v1/webhook/ingest
Autenticación	JWT (Bearer Token)
Entrada	JSON: {"title": "string", "content": "string", "source_url": "string"}
Salida	JSON: {"status": "received", "task_id": "uuid"} (HTTP 200)
Errores	HTTP 401 (No autorizado), HTTP 429 (Rate Limit excedido)

6.2 Modelo de datos y migraciones

Se utiliza una colección NoSQL en Firestore llamada analisis. Cada documento tiene un ID autogenerado (task_id) y contiene los campos: title, result (JSON del LLM), latency_ms, timestamp y status. A continuación en el archivo db.py del proyecto se muestra que la colección guarda un task_id de una variable JSON.

```
def guardar_analisis(task_id: str, data: dict):
    try:
        db.collection("analisis").document(task_id).set(data)
    except Exception as e:
        logger.error(f"Error guardando en Firestore: {e}")
```

En el archivo pipeline.py se utiliza esta variable análisis_data que almacena los valores JSON de la respuesta obtenida del agente Gemini y después se llama a la función guardar_analisis de db.py para almacenar el id y sus metadatos.

```
analisis_data = {
            "article_title": article_data["title"],
            "result": final_response,
            "latency_ms": total_latency,
            "timestamp": time.strftime("%Y-%m-%d %H:%M:%S")
        }

        guardar_analisis(task_id, analisis_data)
```

7. Seguridad, privacidad y cumplimiento

- Autenticación y autorización: El acceso a la API se controla mediante tokens JWT. Las contraseñas se hashean usando passlib.
- Protección: Uso de Secret Manager para inyectar JWT_SECRET_KEY y GCP_PROJECT_ID. Uso de IAM para que la Service Account (media-group-sa) tenga permisos de mínimo privilegio (roles/aiplatform.user, roles/secretmanager.secretAccessor, roles/datastore.user).
- Auditoría: Todos los eventos (ingesta, éxito, error) se registran en Cloud Logging con formato JSON.

8. Observabilidad y operación

- Logs: Implementación de python-json-logger. Se emiten 5 tipos de eventos: WEBHOOK_RECEIVED, PIPELINE_STARTED, LLM_CALL_SUCCESS, CIRCUIT_BREAKER_STATE_CHANGE y PIPELINE_COMPLETED.
- Métricas: El dashboard consulta a Firestore para obtener la latencia promedio y el total de artículos procesados.
- Resiliencia: Implementación de un Circuit Breaker (pybreaker). Si Vertex AI falla 3 veces seguidas, el circuito se abre y el agente devuelve un análisis de respaldo en lugar de colapsar.
- Trafico: Implementación de un limit rating para tener un control de tráfico en solicitudes para evitar problemas en el servidor web y el API.

9. Rendimiento, escalabilidad y costos

- Escalabilidad: Cloud Run escala horizontalmente (a cero si no hay tráfico).
- Estimación de Costos (20,000 artículos/mes):
  - Vertex AI (Gemini 3.5 Flash): ~$35.00 USD.
  - Cloud Run (Compute): $0.00 USD (cubierto por Free Tier).
  - Firestore + Secret Manager: ~$0.15 USD.
- Costo Total: ~$35.15 USD/mes.
- Rendimiento: El modelo de Vertex AI utilizado (Gemini-3.5-flash) proporciona respuestas en segundos, con información detallada y costos económicos.

10. Alternativas consideradas

Alternativa	Ventajas	Desventajas / riesgos	Motivo de aceptación o rechazo
A. Procesamiento Síncrono	Fácil de implementar, Debugging mas sencillo y consume menos recursos.	Timeouts en el webhook. Mala experiencia de usuario. Bloque del flujo que puede congelar las solicitudes del agente.	Rechazado. Se requiere asincronía para una major latencia y control de flujo.
B. LangChain	RAG, memoria integrada en agentes y documentación extensa actualizada.	Menor integración nativa con IAM y Vertex AI ademas de problemas con librerías antiguas.	Rechazado. ADK es nativo de GCP.
C. Ollama Local	Sin costo de API (gratis) y una variedad inmensa de modelos.	Requiere VM con GPU encendida 24/7 (~$500/mes).	Rechazado. El costo de infraestructura supera a Vertex AI.
D. Google ADK + Cloud Run	Escalabilidad serverless, integración nativa, bajo costo.	Requiere configuración de IAM.	Aceptado. Es la solución propuesta.

11. Riesgos y mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación / contingencia	Responsable
Caída de Vertex AI (Error 500)	Media	Alto	Circuit Breaker (PyBreaker) devuelve un	Caída de Vertex AI (Error 500)

			análisis de respaldo.	
Costos elevados por tokens	Baja	Medio	Configuración de alertas de presupuesto en GCP. Limitar longitud del texto.	Costos elevados por tokens

12. Plan de implementación y despliegue

Fase	Actividad	Responsable	Salida / criterio de finalización
1. Preparación	Configurar GCP, habilitar APIs, crear Service Account.	Desarrollador	Proyecto listo en GCP.
2. Desarrollo	Crear Backend (FastAPI, ADK, Firestore) y Frontend (React).	Desarrollador	Código funcional en local.
3. Validación	Pruebas de integración y despliegue en Cloud Run.	Desarrollador	Backend accesible vía URL pública.
4. Despliegue	Configurar Secret Manager y variables de entorno.	Desarrollador	Frontend apuntando a la URL de producción.

13. Pruebas y criterios de aceptación

- Unitarias: Pruebas de la generación de tokens JWT y validación de contraseñas.
- Integración: Envío de artículo desde el Chat y verificación del JSON en Firestore.
- Seguridad: Intento de acceso al webhook sin token (debe devolver 401).
- Monitoreo: Prueba de consulta al agente y revisión del Logger en gcp.

14. Impacto y dependencias

- Dependencias externas: Google Cloud Platform (Vertex AI, Cloud Run, Firestore).
- Impacto en el equipo: El equipo de redacción de MediaGroup ahora puede usar el Chat para analizar artículos en lugar de hacerlo manualmente.
- Documentación: Se utiliza un enlace de github con README.md, Makefile y requirements.txt para realizar una instalación en un nuevo entorno de trabajo.

15. Referencias y anexos

- Mas información de Google ADK: https://adk.dev/
- Mas información de Google Cloud Plataform: https://cloud.google.com/